Exact Gram Matrices from Sparse Row Shifts by Sequential Factor Replacement

Leonard van Hemert

19 October 2026
Technical research artifact, version 1.0.0

Abstract

We study full Gram computation for matrices whose successive rows have sparse differences from a one-coordinate shift. A sequential factor-replacement schedule evaluates a classical displacement-product identity while keeping every Gram accumulator a sum of at most n products of bounded old or new input factors. For $A = 2^e X$, integer $|X_{ik}| \le B$, $nB^2 \le 2^{53}$ and $-400 \le e \le 200$, this gives exact real-valued output in correctly rounded binary64 arithmetic, independently of defect count and recurrence length. The schedule uses two sparse support traversals without merging their supports. Its Gram body uses $mn + 2(P - m) + mQ$ fused multiply-add operations, where $P = m(m+1)/2$ and Q is the total shifted defect count. The implementation validates the dense input and does not quantize it. Lean verifies the algebra, prefix bounds and rounded reconstruction under an explicit primitive rounding model; the machine interpretation and C execution remain separate trust boundaries. We retain all 230 frozen comparisons with Accelerate SYRK and GEMM, including losses. One full-output 768×8192 case takes 9.046 ms versus SYRK's 19.634 ms, a paired speedup of 2.169 with conditional within-run 95% interval $[2.164, 2.175]$. This is a scoped schedule, guarantee and software contribution built from established algebra and bounded-integer arithmetic, rather than a new unrestricted matrix-multiplication exponent or a priority claim.

1 Problem and contribution

Given $A \in \mathbb{R}^{m \times n}$, the target is $G = AA^T$, with either its lower triangle or all m^2 entries materialized. In sliding data, neighboring rows may share most shifted coordinates even when they are not exact windows of one signal. Corrected samples and accumulated sparse innovations are two such models. We exploit the shifted differences, not a low-rank hypothesis.

The central issue is arithmetic order. An exact polynomial identity does not by itself guarantee that its floating-point intermediates are representable. An order that expands both changes against the wrong factor can exceed the capacity of a direct dot product, even when its final answer fits. We complete all left-factor changes against the old right factors, then all right-factor changes against the new left factors. Every prefix retains a bounded-factor interpretation. This supplies an ordinary-dot sufficient capacity bound for sparse reuse, with no dependence on recurrence length.

The contribution is the specified schedule, its complete invariant and admission guarantee, and a validated implementation with retained experimental evidence. The algebra and integer-in-floating-point principle are established. We make no historical-first assertion, universal speedup claim, or claim that dense-library wins establish structured state of the art. Section 6.2 adds a focused structured comparator and separates its new measurements from the original campaign. No results from unrelated research projects are included.

2 Relation to established results

Pan, Murphy and Rosholt's compatible-matrix displacement product rule [3, Section 2.4, Theorem 2.3] has no low-rank prerequisite. Let L be the lower row shift, C an orthogonal cyclic feature shift, and $H = LAC$. Then the elementary product difference

$$AA^T - HH^T = (A - H)H^T + A(A - H)^T \tag{1}$$

is a specialization of that mechanism. The cyclic endpoint terms give the removed head and inserted tail below. Sparse evaluation gives the work mechanism; full residual rank does not make this algebra new.

FFLAS/FFPACK [1, Sections 2.1.2 and 3.2.3] uses bounded integer accumulation on floating-point hardware and analyzes schedule-specific intermediates. We inherit that representability principle. The local extra content is the complete mixed-factor prefix argument for this sparse Gram schedule, retaining $nB^2 \le 2^{53}$ without a defect-count term, together with admission and software realization. This is a concrete elementary specialization.

STOMP [4, Section IV-A, Equation (4)] already gives the zero-defect sliding-dot recurrence. Its matrix-profile output reduces pairwise results rather than materializing a Gram matrix. SCAMP [5, Sections 3.2.4 and 3.3] uses high-precision tile seeds and centered, rearranged updates for numerical stability. Those scheduling and reseeding ideas are not new here. Gilda's recent preprint [2] analyzes periodically re-anchored sliding products under different rounding assumptions; its formal-certification claims were not independently audited for this release. We instead prove zero arithmetic error for the explicit bounded dyadic class with arbitrary row-specific sparse changes.

These sources settle the inherited mechanisms, not universal priority of every implementation detail. The artifact's contribution comparison states each source's hypotheses and conclusions. No absence of a search hit is used as evidence of historical priority. Nor do we compare complete STOMP/SCAMP systems with a different output and validation contract.

3 Construction and real-arithmetic proof

For $i \ge 1$ and $0 \le k < n-1$, put

$$U_{ik} = A_{i-1,k+1}, \qquad D_{ik} = A_{ik} - U_{ik}, \qquad S_i = \operatorname{supp} D_i, \quad q_i = |S_i|, \quad Q = \sum_{i=1}^{m-1} q_i. \tag{2}$$

Set $q_0 = 0$. No mathematical cap on q_i is needed. Compute all first-column anchors $G_{i0} = \sum_{k<n} A_{ik}A_{0k}$. For increasing $i = 1, \dots, m-1$ and $j = 1, \dots, i$, use the following algorithm. Each displayed endpoint or correction operation is one explicit FMA in the C implementation.

```
Validate A; extract ordered nonzero residual lists S_i.
During the same feature traversal, compute first-column anchors.
for i = 1,...,m-1:
    for j = 1,...,i:
        v = G[i-1,j-1]
        v = fma(-A[i-1,0], A[j-1,0], v)
        v = fma( A[i,n-1], A[j,n-1], v)
        for k in S_i: v = fma(D[i,k], U[j,k], v)
        for k in S_j: v = fma(A[i,k], D[j,k], v)
        G[i,j] = v
Reflect the strict lower triangle if full output is requested.
```

The C anchors use eight independent chains followed by a tree over disjoint coordinate subsets. The first correction loop must finish before the second; order within either distinct-coordinate list is arbitrary. Supports can overlap.

Lemma 1 (Complete prefix invariants)**.** *Fix $1 \le j \le i < m$ and an exact predecessor $G_{i-1,j-1}$. After removing the head product and inserting the tail, any prefix $P \subseteq S_i$ of the first correction loop has accumulator*

$$V_1(P) = A_{i,n-1}A_{j,n-1} + \sum_{k<n-1} L_P(k)U_{jk}, \qquad L_P(k) = \begin{cases} A_{ik} & k \in P, \\ U_{ik} & k \notin P. \end{cases} \tag{3}$$

After that loop is complete, any prefix $T \subseteq S_j$ of the second has

$$V_2(T) = A_{i,n-1}A_{j,n-1} + \sum_{k<n-1} A_{ik}R_T(k), \qquad R_T(k) = \begin{cases} A_{jk} & k \in T, \\ U_{jk} & k \notin T. \end{cases} \tag{4}$$

Proof. The predecessor equals its head product plus $\sum_{k<n-1} U_{ik}U_{jk}$. The two endpoint operations therefore give (3) at the empty prefix. Inserting a new support coordinate k adds $(A_{ik} - U_{ik})U_{jk}$, replacing exactly its left factor; induction proves every first prefix. Outside S_i the old and new factors coincide, so the completed first loop equals (4) at $T = \varnothing$. Adding $A_{ik}(A_{jk} - U_{jk})$ replaces exactly the next right factor. This proves every second prefix, including overlapping supports and $i = j$. □

Theorem 2 (Real recurrence and reconstruction)**.** *For arbitrary real A and $n \geq 1$, the ordered update is*

$$G_{ij} = G_{i-1,j-1} - A_{i-1,0}A_{j-1,0} + A_{i,n-1}A_{j,n-1} + \sum_{k \in S_i} D_{ik}U_{jk} + \sum_{k \in S_j} A_{ik}D_{jk}. \tag{5}$$

Exact first-column anchors and these updates reconstruct the entire lower triangle of AA^T for every finite m.

Proof. The final value in (4) is the desired row dot product, proving (5). Its predecessor belongs to an earlier row and is itself lower triangular. Row induction, with the first-column anchors as base cases, proves all required entries. There is no output for $m = 0$. For $n = 0$ define the result as zero separately. At $n = 1$ the support sums are empty; deletion cancels the predecessor's sole product and insertion gives the new product. □

Only distributivity and ordered factor replacement are used: no scalar factors are commuted in the local identity. With appropriate adjoints this also gives an ordered block identity, but no recursive block implementation or speed claim is inferred from it.

4 Exact binary64 execution

Definition 3 (Mathematical admission class)**.** *For $n \geq 1$, let $X \in \mathbb{Z}^{m \times n}$, $B \in \mathbb{Z}_{\geq 0}$ and $e \in \mathbb{Z}$ satisfy*

$$A_{ik} = 2^e X_{ik}, \quad |X_{ik}| \leq B, \quad nB^2 \leq 2^{53}, \qquad -400 \leq e \leq 200. \tag{6}$$

Arithmetic is IEEE binary64 with correctly rounded addition, subtraction, multiplication and explicit FMA, no reassociation changing the schedule, and no flushing of input values during admission. FMA rounds $ab + c$ once.

Lemma 4 (Intermediate coefficient bound)**.** *In coefficient units 2^{2e}, every anchor-chain state and reduction node, the deleted-head state, and every value in (3)–(4) is an integer of magnitude at most nB^2.*

Proof. Each state is a sum of at most n products of actual old or new input coefficients. Each factor has magnitude at most B, so the triangle inequality bounds the sum by nB^2. Anchor chains select disjoint coordinates; tree nodes join disjoint subsets, preserving the same property. Deletion leaves $n-1$ products. Insertion and the two prefix identities give n. □

Theorem 5 (Exact arithmetic)**.** *Under Definition 3, residual subtractions, every Gram anchor operation, both endpoint FMAs and every correction FMA are exact as real operations, normal when nonzero, and finite. The lower output equals AA^T exactly, independently of m, defect count and recurrence length.*

Proof. Every integer of magnitude at most 2^{53} is exactly representable in binary64. An accumulator is $z2^{2e}$ with integer $|z| \leq 2^{53}$ by Lemma 4. Its nonzero magnitude is at least 2^{-800} and its magnitude is at most 2^{453}. Its significant-bit length fits the format, and the exponent range is safely normal and finite. Thus its exact real value is representable; correct rounding returns it unchanged. Induction over each chain, tree node and update, and then over predecessor rows, proves exactness.

For residuals, $n \geq 1$ implies $B < 2^{27}$, so each difference has integer coefficient of magnitude at most $2B < 2^{28}$ at scale 2^e. It is representable, normal when nonzero and finite. If a residual product exists then $n \geq 2$ and its coefficient is at most $2B^2 \leq 2^{53}$; at $n = 1$ none occurs. Hence the subtractions and their operands are exact as well. No premise assumes an exact intermediate or an exact anchor: those properties were derived. □

This proves zero real error, including cancellation zeros and nearly dependent rows. It is stronger than the campaign's fixed requirement $|\widehat{G}_{ij} - G_{ij}| \leq 2 \cdot 10^{-12}\|A_i\|_2\|A_j\|_2$. Signed-zero bit patterns are not identified with a unique real encoding. The square-root estimate used to select an admission cap is not asserted to be an exact real square root; integer comparisons correct its result.

4.1 A strict grouped-order counterexample

The alternative grouped corrections $D_iA_j^T + U_iD_j^T$ have the same final polynomial but do not inherit the ordinary-dot bound automatically. Take $n = 2$, $e = 0$, $B = 2^{26}$ and rows $[0, -B]$, $[B-1, B]$. There is one defect, and $nB^2 = 2^{53}$. Both orders reach $2B^2$ after the endpoint operations. The grouped first correction requests

$$2B^2 + (2B-1)(B-1) = 4B^2 - 3B + 1 = 18014398308155393,$$

which round-to-nearest ties-to-even changes to 18014398308155392. Its final answer is 9007199120523264, whereas the exact, representable diagonal is $(B-1)^2 + B^2 = 9007199120523265$. The sequential order first computes $2B^2 + (2B-1)(-B) = B = 67108864$, then returns the exact diagonal. The independent verifier executes these FMAs. This is a strict comparison with that local grouped order, not a lower bound against every published algorithm.

5 Implementation contract and cost

The public routine writes the lower triangle of a row-major, strided double matrix. Inputs and outputs must be disjoint valid allocated objects. Its implementation restrictions, separate from the mathematical theorem, are $n \le 65536$, `max_q`≤ 64, and $-400 \le e \le 200$. For nonempty input it uses

$$B_n = \left\lfloor \sqrt{\lfloor 2^{53}/n \rfloor} \right\rfloor .$$

The quotient and final square tests use integer arithmetic; a floating square root provides only an initial estimate. For every actual input it checks finiteness, the cap, $x = A_{ik}2^{-e} = \mathrm{trunc}(x)$ and $x2^e = A_{ik}$. The last check rejects tiny invalid values that scale to zero. No projection or quantization is performed. The shifted support cap is checked before every residual store. Validation, exact residual extraction and eight anchor chains share a feature traversal; NEON combines predicates over eight values before branching. A scalar tail of one through eight values prevents shifted loads beyond the row.

Status zero certifies exact output. Negative codes report invalid pointers, strides or extents (-1), nonfinite input or excess defects (-2), allocation failure (-3), or unsupported lattice/parameter class (-4). Error-code precedence for multiply-invalid inputs can differ between scalar and vector paths; both reject. Discard partial output on error. Extents and allocation products are checked before use. At $m = 0$ the routine returns without accesses. At $m > 0, n = 0$ it validates output extents and parameter caps, then writes lower zeros without reading A. At $n = 1$ residual storage is absent. There is no automatic fallback to a different numerical domain.

Proposition 6 (Gram work and requested workspace)**.** *For $m, n > 0$, put $P = m(m+1)/2$ and $D = P - m$. The Gram body uses exactly*

$$F = mn + 2D + mQ \tag{7}$$

FMAs. With $c = \min(\texttt{max_q}, n-1)$, its requested heap workspace on the recorded 64-bit ABI is $16(m-1)c + 8(m+1)$ bytes if $(m-1)c > 0$, and zero otherwise.

Proof. Anchors use mn FMAs, and the D recurrence entries use two endpoint FMAs each. Residual row $t \ge 1$ occurs t times in the first correction role and $m - t$ times in the second, thus exactly m times. This proves (7), including both correction occurrences on diagonal entries. The implementation reserves $(m-1)c$ records of one eight-byte index and one eight-byte residual, and $m+1$ eight-byte offsets only when capacity is positive. No norm/error array, packed dense input, second Gram buffer or repair dot is allocated. □

Admission separately performs $2mn$ scaling multiplications and predicates; source-level anchor reductions add $7m$ additions, and extraction uses Q subtractions. Thus (7) is not a complete instruction count. At $n = 1$ this schedule uses m^2 FMAs, and need not be efficient. At $n = 0$ it performs no FMAs; the nonempty work formula is not applied to that branch.

The caller still stores $8mn$ input bytes and $8m^2$ output bytes. At 768×8192, cap two, requested scratch is 30,696 bytes versus a 48 MiB input and 4.5 MiB output. With two defects in every later row, the Gram body has 8,058,624 FMAs, compared with 2,419,064,832 classical SYRK product terms, but admission adds 12,582,912 multiplications.

A reuse-aware preparation operand model is $8[2mn+(m-1)(n-1)]$ bytes, plus up to 128 source-operand bytes per changed eight-value block if scalar extraction rereads all its lanes. Recurrence output stores and predecessor reads together demand $8m^2$ logical bytes; endpoints, correction factors and residual records

Family	Shape	q	e	SYRK ms	GEMM ms	Seq. ms	SYRK/seq. [95%]
Innovations	768×8192	2	-15	19.6336	20.5861	9.0460	2.169 [2.164, 2.175]
Innovations	768×8192	4	0	30.4593	31.1561	15.2613	2.045 [1.889, 2.229]
Windows	512×4096	2	-15	3.2515	3.6209	2.0573	1.596 [1.554, 1.612]
Innovations	512×4096	2	-15	2.9476	3.3151	1.9707	1.541 [1.438, 1.588]
Innovations	384×2048	4	-15	1.2021	1.3621	1.2632	0.952 [0.943, 0.982]
Innovations	192×768	2	-15	0.0820	0.0858	0.1353	0.612 [0.606, 0.614]

Table 1: Selected original full-output runs, uniform signed 16-bit coefficients, seed 20261201. Times are method medians; speedups are paired-ratio medians. The final two rows retain losses. Intervals describe within-run variability.

add $32D$, $8mQ$ and $16mQ$ bytes before reuse. Offset accesses and control add costs. Full reflection adds $16D$ logical bytes. These are transparent source-access models, not measured DRAM traffic or retired-load counts. Cached rows and register hoisting can remove physical transfers. Vendor internal workspace and total process peak memory were not instrumented. Small-case losses below demonstrate that product counts alone do not predict wall-clock speed.

6 Experimental evidence

6.1 Original frozen campaign

The original 230 runs and every individual sample are retained unchanged. The candidate was fixed as method 13 before reserved evaluation. Each run compares Accelerate DSYRK, native full DGEMM, an earlier general-real certified path, grouped lattice updates, merged-support substitution and the sequential member. The two other lattice members are local structured comparators, not executions of all published structured algorithms.

All methods receive the same dense row-major binary64 input and output layout, precision, thread setting and output contract. On Apple M4, 16 GiB, macOS 27.2 build 26B5091g, Apple clang 21, the compile flags are `-O3 -std=c11 -fno-fast-math -ffp-contract=off`. Single-thread Accelerate is selected and checked through its threading API; `VECLIB_MAXIMUM_THREADS=1` is also set. The ordinary macOS scheduler is used without affinity. These are single-thread claims, not comparisons against the fastest multithread configuration. The run used battery power (39% to 31%); a final query reported no thermal/performance warning, not continuous thermal monitoring.

The per-call timer includes validation, extraction, anchors, temporary allocation/free, the complete update and, for full output, a shared blocked 32×32 reflection. Native GEMM already writes full output and is not redundantly reflected. There are five warmups per method, calibration to at least 10 ms, and 30 paired samples in five randomized balanced cyclic blocks of six methods. Operands are warm and repeated. Source/binary hashes are checked between runs. Correctness work is separate from timing.

The 180 reserved uniform-innovation runs use shapes 192×768, 384×2048, 768×8192, caps 1/2/4, exponents $0/-15$, five seeds 20261201–20261205, and lower/full output. Fifty additional full-output runs use 512×4096, cap two, $e = -15$, corrected windows or accumulated innovations, and five distributions: uniform signed 16-bit, nearly constant integer, saturated alternating, saturated impulse and ternary coefficients. These are 230 comparison runs, not 230 independent draws: lower/full and scales reuse coefficient data, and deterministic window patterns repeat.

Before timing, poisoned output and an all-entry finiteness scan detect missing writes. An independent int64 oracle checks the first 128 diagonals and 256 seeded pairs, demanding exact equality for lattice members and the fixed norm-scaled tolerance for other methods. Full reflected outputs are checked for symmetry; full GEMM samples cover both triangles. The signed 16-bit coefficient sums have absolute value at most 2^{46}, so the oracle's integer sum and scaled conversion are exact. This is sampled preflight, not an oracle for every entry of every timed matrix.

Table 1 reports the documented 2.169 speedup; the paired GEMM speedup for that row is 2.277 with interval $[2.272, 2.282]$. All 60 runs at 768×8192 and all 50 distribution runs at 512×4096 beat both vendors. All 60 at 192×768 lose; at 384×2048 cap four loses to SYRK whereas caps one and two win. There are 150 paired-median wins against both vendors, also 150 with both conditional interval lower endpoints above one. Every result is available without rerunning timings.

Intervals resample the five contiguous balanced blocks 5,000 times, taking the median paired ratio. They have only five blocks and are conditional within-run summaries, not cross-machine portability guarantees

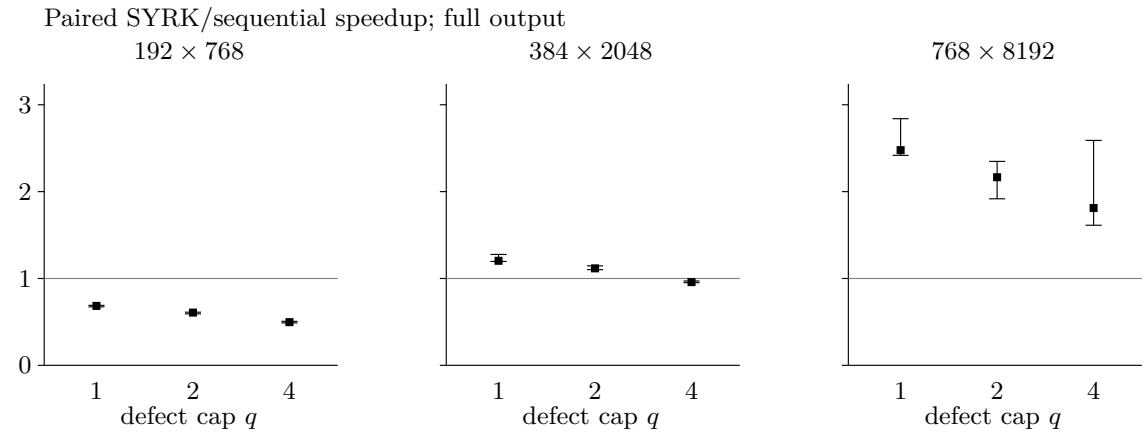


Figure 1: Original full-output uniform-innovation groups. Each marker is the median of ten run-level paired speedups (two scales, five seeds); bars show the observed minimum and maximum, not confidence intervals. The gray line is parity with single-thread SYRK.

or simultaneous coverage of 230 comparisons. Both vendors are reported; choosing a vendor by run-median time is not a per-trial fastest-vendor oracle.

6.2 Focused structured comparator

For a same-contract alternative based on known ingredients, reconstruct a Hankel generator h from the actual first row and later row tails: $h_k = A_{0k}$ for $k < n$ and $h_{n-1+i} = A_{i,n-1}$ for $i \geq 1$. Set $H_{ik} = h_{i+k}$ and $E = A - H$. A standalone comparator computes HH^T using the classical sliding recurrence, then applies

$$AA^T = HH^T + EH^T + AE^T$$

with all left replacements preceding the right replacements. Its factors come from actual input entries, so the same bounded-factor proof gives exactness. It validates the lattice and local shifted cap, counts and allocates actual E, and includes generator reconstruction, all scans, corrections, allocation and full reflection in the timed call. It receives no latent generator or trusted edit locations. This is an implementation derived here from established ingredients, not an asserted published implementation.

Its correction count depends on $R = \mathrm{nnz}(E)$ rather than Q: $mn + 2D + (m+1)R$ Gram FMAs (corrections include the first column). Corrected windows may have small R, whereas accumulated innovations can make R large even when Q is small. The comparison is useful precisely because it exposes that structural distinction. It is a focused baseline, not a comprehensive structured-library contest.

The new 20-run campaign is stored separately from the original 230. Before timing it freezes 512×4096, cap two, $e = -15$, full output, both families, uniform/nearly constant coefficients and the same five seeds. Four methods (SYRK, GEMM, sequential, reconstructed-Hankel) receive 32 paired samples in eight balanced blocks of four, with five warmups and 10 ms calibration. Independent generator fingerprints verify seed and input identity; a source/binary-bound manifest detects missing, duplicated or inconsistent jobs. No original measurement is replaced. The sequential member beats both dense vendors and this

Family	Coefficients	R range	SYRK/seq.	H+E/seq.
Innovations	Uniform	229917–234577	1.352 [1.337,1.381]	45.831 [44.659,46.299]
Innovations	Near constant	213176–219293	1.365 [1.355,1.406]	42.544 [42.010,43.542]
Windows	Uniform	518	1.575 [1.545,1.583]	1.746 [1.729,1.757]
Windows	Near constant	518	1.581 [1.531,1.584]	1.741 [1.712,1.777]

Table 2: Separate new structured comparison: medians of five run-level paired ratios with observed minimum/maximum in brackets, not confidence intervals. All jobs use 512×4096, cap two, $e = -15$, full output. R counts actual reconstructed Hankel corrections.

reconstructed-Hankel implementation in all 20 new runs. It is about 1.7–1.8 times faster than the latter on these corrected windows, and about 42–46 times faster on accumulated innovations. The large latter ratio reflects the growth of reconstructed corrections, not a claim against every structured method. An optimized alternative Hankel reconstruction or a different published structured implementation remains an outstanding comparison. No structured state-of-the-art claim follows.

6.3 Independent numerical validation

The portable strict native/scalar verifier checks 2,044 lower entries per implementation against Python integer products with exact Fraction scaling. It covers short vector tails, individual defect lanes, cap and exponent endpoints, overlaps, zero rows, cancellation, padded strides, q=0/64, empty dimensions and invalid inputs. Additional independent finite checks cover 6,561 states, an ordered polynomial and 371 CSR-prefix states. These finite tests supplement the proof rather than replace it.

A separate integer-first generator and direct int64 oracle check every entry of four representative large matrices in contiguous and padded layouts. The counts are 1,263,744 candidate entries, 1,263,744 SYRK entries and 2,523,136 GEMM entries: 5,050,624 comparisons in total, all exactly equal. No nonfinite, missing-write, input or padding violation occurs. The vendors therefore also achieve zero error on these cases. The total must not be attributed solely to the candidate. Fresh release tests and original receipts are retained separately.

7 Formalization and trust boundary

The pinned project uses Lean 4.24.0 and mathlib revision `f897ebcf72cd16f89ab4577d0c826cd14afaafc7`; its dependency manifest pins all transitive revisions. A successful project build and exact-type plus `#print axioms` output for 53 declarations are retained. Only `propext`, `Classical.choice` and `Quot.sound` occur in the inspected closures; there is no `sorryAx` or custom mathematical axiom. The artifact maps every principal declaration to its exact statement.

Manuscript statement	Principal Lean declarations in namespace `GramExact`
Lemma 1	`first_prefix`, `first_step`, `second_prefix`, `second_step`
Theorem 2	`recurrence`, `reconstruction`, `groups_complete`
Lemma 4	`first_prefix_bound`, `second_prefix_bound`, `product_subset_bound`
Theorem 5, abstract model	`rounded_residual_exact`, `rounded_groups_exact`, `rounded_gram_exact`, `binary64_gram_exact`
Proposition 6, nonempty FMA count	`support_work`, `total_work`

Table 3: Formal mapping. Workspace bytes, C validation, compiler execution and experimental times are outside these declarations.

The formal arithmetic model makes the machine boundary explicit. Define

$$\mathcal{D}(s) = \{zs : z \in \mathbb{Z},\ |z| \le 2^{53}\}.$$

A `RoundingModel` supplies a function $r : \mathbb{R} \to \mathbb{R}$ that fixes $\mathcal{D}(s)$. Rounded FMA is $r(ab+c)$. Integer prefix bounds and ring identities derive primitive and algorithmic exactness; no Gram conclusion, exact anchor or exact prefix is postulated. Constructed rounded anchors discharge the intermediate reconstruction theorem's anchor premise.

`Binary64Bridge` assumes only that the interpreted rounding fixes $\mathcal{D}(2^a)$ for $-1022 \le a \le 970$. This subset is analytically representable in binary64: coefficients have at most 53 significant bits (the endpoint 2^{53} is itself a power of two), minimum nonzero exponent -1022, and maximum leading exponent $970+53 = 1023$. Lean proves the input and doubled exponent containment and transports the abstract rounded reconstruction to the real Gram target. It does not prove a bit-level IEEE encoding theorem or instantiate a hardware rounding function. This primitive correspondence remains an exposed hypothesis, not an axiom asserting the desired algorithm conclusion.

The final theorem admits arbitrary natural feature count, including zero, unbounded row indices restricted by $j \le i$, integer coefficients and the admission bound. A finite matrix is represented by zero extension. Lean uses $N = 0$ or $N = n+1$ to handle the tail coordinate. Its folds accept every distinct support enumeration; a `foldr` is matched to the increasing C scan by reversing its supplied list. The canonical formal anchors are a direct chain, and a separate disjoint-merge lemma supports the C eight-chain tree. Residual subtraction exactness is a separate proved bridge to the scaled coefficient differences used by the formal update.

These are mathematical correspondence arguments. Lean does not verify the C program, NEON loads, allocation safety, compiler, signed-zero representation, timing harness or wall-clock measurements. Source audits, strict tests and retained receipts provide separate evidence for those delivered artifacts. The trust

base includes the Lean kernel/distribution and pinned imported libraries; cached dependencies were used rather than rebuilding all mathlib.

8 Artifact availability and boundaries

The self-contained repository is `https://github.com/LeonardSEO/exact-sparse-gram`. It includes this source and compiled paper, the unchanged frozen C member and API, pinned Lean sources, portable tests, optional Accelerate checks, all original measurements, the separate structured comparison, analysis and vector figure sources, claim map, provenance and checksum manifest. Recorded evidence is usable without rerunning the performance campaign. The version tag and publication receipts identify the immutable source commit and any verified Zenodo version DOI. Citation metadata updates after DOI registration do not move that tag.

Code and Lean sources are MIT licensed; the manuscript, figures, documentation and retained data are CC BY 4.0, as confirmed by the author. Third-party dependencies retain their own licenses and are fetched from pinned sources; dependency caches and external papers are not redistributed. Computational assistant use is disclosed in the provenance; no assistant is listed as an author.

The demonstrated speed regime is structured, bounded, warm, single-thread FP64 on the documented machine. No claim covers arbitrary dense matrices, all support placements, other machines, cold-cache operation, GPUs, energy or application throughput. Off-grid inputs are rejected rather than altered. The measured dense-library advantage is distinct from specialized comparisons. The work therefore supplies a precise, checkable execution schedule and exactness guarantee with a scoped implementation result, while preserving the limitations and losing cases needed to interpret that result.

References

[1] Jean-Guillaume Dumas, Pascal Giorgi, and Clément Pernet. Dense linear algebra over finite fields: the FFLAS and FFPACK packages. *ACM Transactions on Mathematical Software*, 35(3):19, 2009. URL `https://arxiv.org/abs/cs/0601133v3`. Author manuscript arXiv:cs/0601133v3; Sections 2.1.2 and 3.2.3.

[2] Sankalp Gilda. Forward error analysis of periodically re-anchored sliding-window inner-product recurrences, with application to the matrix profile, 2026. URL `https://zenodo.org/records/20599316`. Preprint, version v1, published 8 June 2026, modified 29 June 2026.

[3] Victor Y. Pan, Brian Murphy, and Rhys E. Rosholt. Unified nearly optimal algorithms for structured integer matrices and polynomials. Technical Report TR-2008003, Graduate Center, City University of New York, 2008. URL `https://academicworks.cuny.edu/gc_cs_tr/308/`. 1 May author manuscript, Section 2.4, Theorem 2.3.

[4] Yan Zhu, Zachary Zimmerman, Nader Shakibay Senobari, Chin-Chia Michael Yeh, Gareth Funning, Abdullah Mueen, Philip Brisk, and Eamonn Keogh. Matrix profile II: Exploiting a novel algorithm and GPUs to break the one hundred million barrier for time series motifs and joins. In *IEEE International Conference on Data Mining*, 2016. URL `https://mcyeh.github.io/paper/2016_icdm_exploiting_novel_algorithm.pdf`. Section IV-A, Equation (4).

[5] Zachary Zimmerman, Kaveh Kamgar, Nader Shakibay Senobari, Brian Crites, Gareth Funning, Philip Brisk, and Eamonn Keogh. Matrix profile XIV: Scaling time series motif discovery with GPUs to break a quintillion pairwise comparisons a day and beyond. In *Proceedings of the ACM Symposium on Cloud Computing*, 2019. doi: 10.1145/3357223.3362721. URL `https://www.cs.ucr.edu/~eamonn/SCAMP-camera-ready-final1.pdf`.